

Ayudantía N°4 Teoría

Felipe Llach R.

07-05-2026

1. Patrón Singleton (Creacional)

1.1. Definición

El patrón **Singleton** garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella. Es uno de los patrones más sencillos.

1.2. Estructura y Funcionamiento

Para implementar un Singleton, se deben cumplir tres reglas fundamentales:

- **Constructor Privado:** Evita que se creen instancias mediante el operador **new** desde fuera de la clase.
- **Atributo Estático Privado:** Almacena la única instancia de la clase.
- **Método/Propiedad Estática Pública:** Actúa como el único punto de entrada para obtener la instancia.

1.3. Casos de Uso Comunes

- **Gestión de Configuraciones:** Cuando toda la aplicación debe compartir los mismos parámetros globales.
- **Pool de Conexiones:** Para limitar y reutilizar conexiones a una base de Datos.
- **Sistemas de Registro (Logging):** Para asegurar que todos los mensajes de error se escriban de forma centralizada.

2. Patrón Composite (Estructural)

2.1. Definición

El patrón **Composite** permite tratar a objetos individuales y a composiciones de objetos de manera uniforme. Facilita la creación de estructuras jerárquicas tipo árbol.

2.2. Componentes Principales

- **Componente** Declara la interfaz común para todos los objetos de la estructura.
- **Hoja:** Representa los objetos finales que no tienen hijos. Implementan el comportamiento base.
- **Compuesto:** Almacena sub-componentes (hojas u otros compuestos) e implementa los métodos de la interfaz delegando el trabajo a sus hijos.

2.3. Casos de Uso Comunes

- **Sistemas de Archivos:** Donde una carpeta (compuesto) contiene archivos (hojas) y otras carpetas.
- **Interfaces Gráficas (GUI):** Un panel que contiene botones, etiquetas y otros paneles.
- **Árboles de Menú:** Menús que se despliegan en submenús.

2.4. Ventajas

Permite que el cliente ignore la diferencia entre objetos individuales y colecciones de objetos, simplificando enormemente el código.